[CTF WRITEUP]

RUST BINARY CRACKME

Reverse Engineering a Password-Protected Login Binary

Category: Reverse Engineering // Difficulty: Beginner-Intermediate

1. Overview

In this challenge, we are given a compiled Rust binary. Running it reveals a simple password prompt. Our goal is to reverse engineer the binary to find the correct password and retrieve the flag — without ever knowing the password in advance.

This writeup walks through the complete solve path step-by-step, from first execution through to extracting and decoding the flag. Each step uses Binary Ninja as our primary tool.

2. Run the Binary

The first thing you should always do before opening a binary in a disassembler is simply run it and observe its behaviour. This tells you what kind of program you're dealing with, what inputs it expects, and what success/failure looks like.

```
Enter password:
blahblahblah
Access denied!
```

Figure: Running the binary with a wrong password

From this we learn three things immediately:

- The program prompts for a password
- Entering a wrong password prints "Access denied!"
- There must be a corresponding "Access granted" path somewhere in the binary — that's our target

3. Open in Binary Ninja & Find the Anchor String

Open the binary in Binary Ninja. The first instinct for many beginners is to open the String List (View > String List) and search for the password. This is the wrong approach for a well-designed challenge — there may be thousands of strings, and the password won't be human-readable anyway.

The correct approach is to anchor to a string you already know exists: "Access denied" or "Access granted". Search for this in the String List instead.

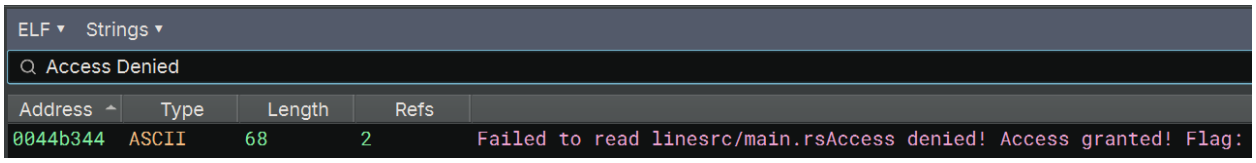

Figure: Searching for 'Access Denied' in the String List reveals the key string cluster

Notice that Binary Ninja shows all the program's key strings concatenated at address

`0044b344`: "Failed to read line", "src/main.rs", "Access denied!", "Access granted! Flag:". This is Rust's string storage — it packs related strings together in the binary's data section.

4. Follow Cross References

With the string selected, open the Cross References panel. In Binary Ninja you can do this by clicking on the line with the anchor string.

The Cross References panel shows two sections:

- Data References — the 4 entries showing where these strings live in memory as char arrays of different sizes
- Code References — the 2 entries showing where in the actual executed code these strings are loaded

```
Cross References
Filter (6)
Data References                                          {4}
  |← 0045e010 char const (* data_45e010)[0x31]
  |← 0045e028 char const (* data_45e028)[0x26]
  |← 0045e038 char const (* data_45e038)[0x17]
  |← 0045e048 char const (* data_45e048)[0x1]
Code References                                          {2}
  Rust::main::h955fa86c44051e1e                          {2}
    |← 00409855 lea     rdx, [rel 0x44b344]
    |← 004098f7 lea     rdi, [rel 0x44b344]
```

Figure: Cross References panel showing 2 code references to our string cluster

The two Code References inside Rust::main are what we care about:

- 0x00409855 — lea rdx, [rel 0x44b344] *loads the string address into rdx (second argument)*
- 0x004098f7 — lea rdi, [rel 0x44b344] *loads the string address into rdi (first argument)*

Double-click the second reference at 0x004098f7 to jump into the main function's logic. This puts us right in the area where the password comparison result is used to decide which string to print.

\

5. Finding the stored, correct password

After jumping to the cross reference, you will land right under the if statement that checks if the input is equal to the password. We can now see that the password is stored at data_44b01.

```
004097d7        if (_mm_movemask_epi8(__pcmpeqb_xmmdq_memdq(*r15_1, data_44b010)) != 0xffff)
004097d7            goto label_4097e0
004097d7
00409864        _$LT$alloc..string..Stri...GT$$GT$::from_iter::hacb14b6ccc9e948a(&var_58,
00409864            &data_44b330,
00409864            "Failed to read linesrc/main.rsAccess denied!\nAccess granted! Flag: \n")
```

Figure: The "if" statement that checks input against stored password

Double clicking this will bring you to where the password is stored

```
0044b010  int128_t data_44b010 =
0044b010  1a 2b 3e 3e 21 6b 79 69-1a 79 6b 21 7b 21 69 79  .+>>!kyi.yk!{!iy
```

Figure: The stored password

Now, if we try to enter this as the password it still fails. This means that there must be something that scrambles your input to be compared with the scrambled password.

6. Identify the XOR Scramble Function

Going back to the disassembled main function, we can see right above the if statement where we found the scrambled password that there is a XOR cipher.

```
004096ca        r15_1 = rax_4
004096cd        int64_t rax_5 = 0
004096cd
004096e6        if (size_1 u>= 0x20 && r15_1 - rbx_3 u>= 0x20)
004096ed            rax_5 = size_1 & 0xffffffffffffffe0
004096f1            rbx_3 = &rbx_3[rax_5]
004096fc            int128_t* rcx_4 = &var_70[0x10] + arg2
004096ff            int64_t rdx_2 = 0
00409701            char zmm0_1[0x10] = *"ZZZZZZZZZZZZZZZZ"
00409701
00409737            do
0040971f                int128_t zmm2_1 = *(rcx_4 + rdx_2) ^ zmm0_1
00409723                *(r15_1 + rdx_2) = *(rcx_4 + rdx_2 - 0x10) ^ zmm0_1
00409729                *(r15_1 + rdx_2 + 0x10) = zmm2_1
00409730                rdx_2 += 0x20
00409737            while (rax_5 != rdx_2)
00409737
0040973e            if (size_1 == rax_5)
0040973e                goto label_4097e0
0040973e
00409757        void* rcx_7 = not.q(rbx_3) + r14_1
0040975a        uint64_t i_1 = zx.q(r14_1.d - rbx_3.d) & 3
0040975a
0040975d        if (i_1.d != 0)
00409774            uint64_t i
00409774
00409774            do
00409760                char rsi_15 = *rbx_3
00409763                rbx_3 = &rbx_3[1]
0040976a                r15_1[rax_5] = rsi_15 ^ 0x5a
0040976e                rax_5 += 1
00409771                i = i_1
00409771                i_1 -= 1
00409774            while (i != 1)
00409774
0040977a        if (rcx_7 u>= 3)
004097b9            do
00409786                r15_1[rax_5] = *rbx_3 ^ 0x5a
00409791                r15_1[rax_5 + 1] = rbx_3[1] ^ 0x5a
0040979d                r15_1[rax_5 + 2] = rbx_3[2] ^ 0x5a
004097a9                r15_1[rax_5 + 3] = rbx_3[3] ^ 0x5a
004097ae                rax_5 += 4
004097b2                rbx_3 = &rbx_3[4]
004097b9            while (rbx_3 != r14_1)
004097b9
004097bf        if (rax_5 != 0x10)
004097bf            goto label_4097e0
```

Figure: The XOR scramble loop — input bytes are XOR'd with 0x5A before comparison

There are three important things to observe in this section of code:

The XOR Key

At address `00409701` you see:

```
char zmm0_1[0x10] = *"ZZZZZZZZZZZZZZZZ"
```

The character 'Z' has ASCII value `0x5A`. This is loading 16 copies of `0x5A` into a SIMD register — this is your XOR key.

The Main XOR Loop

The bulk processing loop (handling 4 bytes at a time) is visible at addresses 0x00409786 through 0x004097a9:

```
r15_1[rax_5]     = *rbx_3     ^ 0x5a
r15_1[rax_5 + 1] = rbx_3[1]  ^ 0x5a
r15_1[rax_5 + 2] = rbx_3[2]  ^ 0x5a
r15_1[rax_5 + 3] = rbx_3[3]  ^ 0x5a
```

This is unambiguous: every byte of the user's input is XOR'd with 0x5A. The result is stored in a new buffer (r15_1), which is then compared to the stored scrambled password.

Why XOR is Reversible

XOR is its own inverse. If the program does:

```
scrambled = input XOR 0x5A
```

Then to reverse it, you simply do the same operation again:

```
input = scrambled XOR 0x5A
```

This means once we find the stored scrambled bytes, we can recover the original password trivially.

7. Decode the Password

We now have everything we need. The decode step is straightforward — XOR each byte with 0x5A. You can do this in the Binary Ninja Python Console (View > Python Console) without leaving the tool:

```
data = [0x1a, 0x2b, 0x3e, 0x3e, 0x21, 0x6b, 0x79, 0x69,
        0x1a, 0x79, 0x6b, 0x21, 0x7b, 0x21, 0x69, 0x79]

print(''.join(chr(b ^ 0x5A) for b in data))
```

Or using Binja's API to read the bytes directly from the binary:

```
data = bv.read(0x44b010, 16)
print(''.join(chr(b ^ 0x5A) for b in data))
```

Running either of these will print the plaintext password. Enter it into the running binary to receive the flag.

8. Summary — The Full Solve Path

Here is the complete solve path condensed:

1. Run the binary to understand what it does
2. Search String List for 'Access granted' — not for the password
3. Double-click the string result to jump to its location
4. Open Cross References (X) and jump to the Code Reference in main
5. Scroll up from the second reference to find the checking if statement
6. Find the SIMD comparison — note data_44b010 as the stored target
7. Double-click data_44b010 to view the raw scrambled password bytes
8. Find the XOR loop — spot the ^ 0x5a operations on every byte
9. XOR each byte with 0x5A in the Python console to recover the plaintext
10. Enter the password to get the flag

Key Concepts Demonstrated

- Anchoring — always start from known strings, not unknown ones
- Cross References — tracing data usage back to code
- HLIL view — Binja's high-level representation makes Rust readable
- XOR cipher — recognising ^ 0x5A as a reversible byte transformation
- SIMD comparison — understanding _pcmpeqb / _mm_movemask_epi8 in context
- Compiler unrolling — the 4-bytes-at-a-time loop is compiler optimization, not a complex algorithm